5.Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the

following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) = (ap + b)

mod 26 A basic requirement of any encryption algorithm is that it be one-to-one. That is, if p q, then

E(k, p) E(k, q). Otherwise, decryption is impossible, because more than one plaintext character maps into

the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example,

for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3.

a. Are there any limitations on the value of b?

b. Determine which values of a are not allowed.

Aim:

To implement the affine Caesar cipher using the formula \(C=(ap+b)\bmod 26\).

Procedure:

1. Read the plaintext and values of a and b.
2. Check that a is relatively prime to 26.
3. Convert each letter to a number from 0 to 25.
4. Apply the affine encryption formula.
5. Display the ciphertext.

Code:

```
#include <stdio.h>
#include <ctype.h>
int gcd(int a, int b) {
   while (b != 0) {
      int t = b;
      b = a % b;
      a = t;
   }
   return a;
}
```

```
int main() {
    char text[100];
    int a, b, i;
    printf("Enter plaintext: ");
    fgets(text, sizeof(text), stdin);
    printf("Enter a and b: ");
    scanf("%d%d", &a, &b);
    if (gcd(a, 26) != 1) {
        printf("Invalid value of a");
        return 0;
    }
    for (i = 0; text[i]; i++) {
        if (isalpha((unsigned char)text[i])) {
            char base = isupper(text[i]) ? 'A' : 'a';
            text[i] = ((a * (text[i] - base) + b) % 26) + base;
        }
    }
    printf("Ciphertext: %s", text);
    return 0;
}
```

Input:

Plaintext: HELLO

a = 5

b = 8

Output:

